



Introducción a la Programación Orientada a Objetos

DCIC - UNS

2025



PROYECTO GUI

IPOO Resto

El objetivo del proyecto es completar la implementación de una aplicación con una interfaz gráfica de usuario (GUI) que facilite la gestión de pedidos en un restaurante. Esta aplicación permitirá a los usuarios ocupar y liberar mesas, consultar y agregar pedidos a cada mesa y ver el estado de los combos disponibles en el menú.

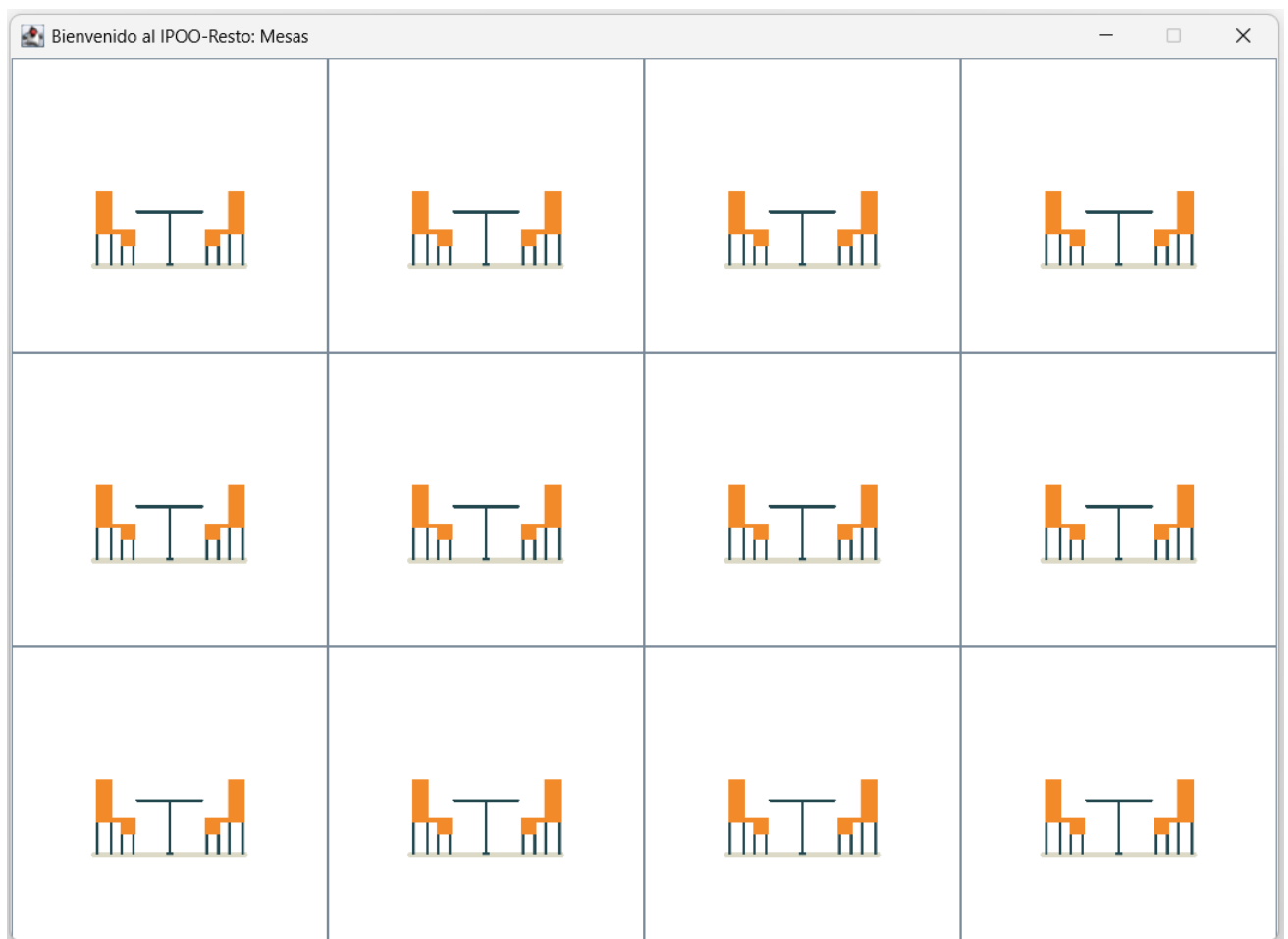
Funcionalidades requeridas de la aplicación:

1. **Ocupación de Mesas:** El usuario debe poder seleccionar una mesa y ocuparla. Al ocupar una mesa, se crea un pedido vacío asociado a esa mesa y cambia su estado de libre a ocupada.
2. **Gestión de Pedidos:** Seleccionada una mesa ocupada, el usuario puede agregar combos a su pedido. Cada vez que se agrega un combo se actualiza el detalle parcial del pedido y el total a pagar. Si se alcanza el máximo de combos permitidos por mesa, se deshabilita el botón para agregar más.
3. **Liberación de Mesas:** Al finalizar el pedido, el usuario puede liberar la mesa. Se muestra un ticket con el resumen del pedido y el total a pagar y se restablece el estado de la mesa a libre.
4. **Gestión del Menú:** La aplicación debe ofrecer una interfaz gráfica que muestre todos los combos disponibles. Cada combo debe mostrar su nombre, descripción, precio y la cantidad disponible. Al vender un combo se debe actualizar el stock y, cuando llegue a cero, deshabilitar su venta.

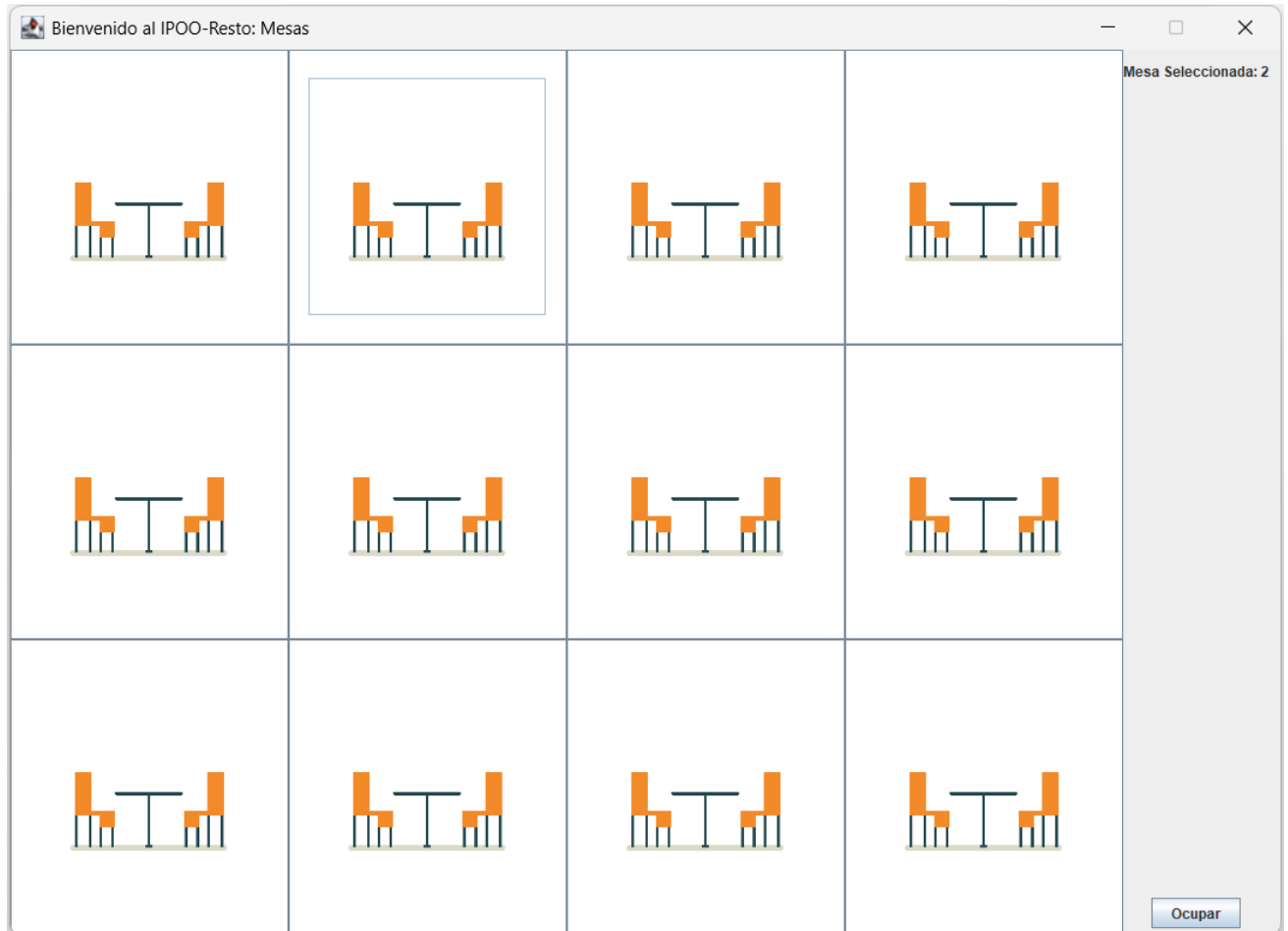


¡Manos a la obra!

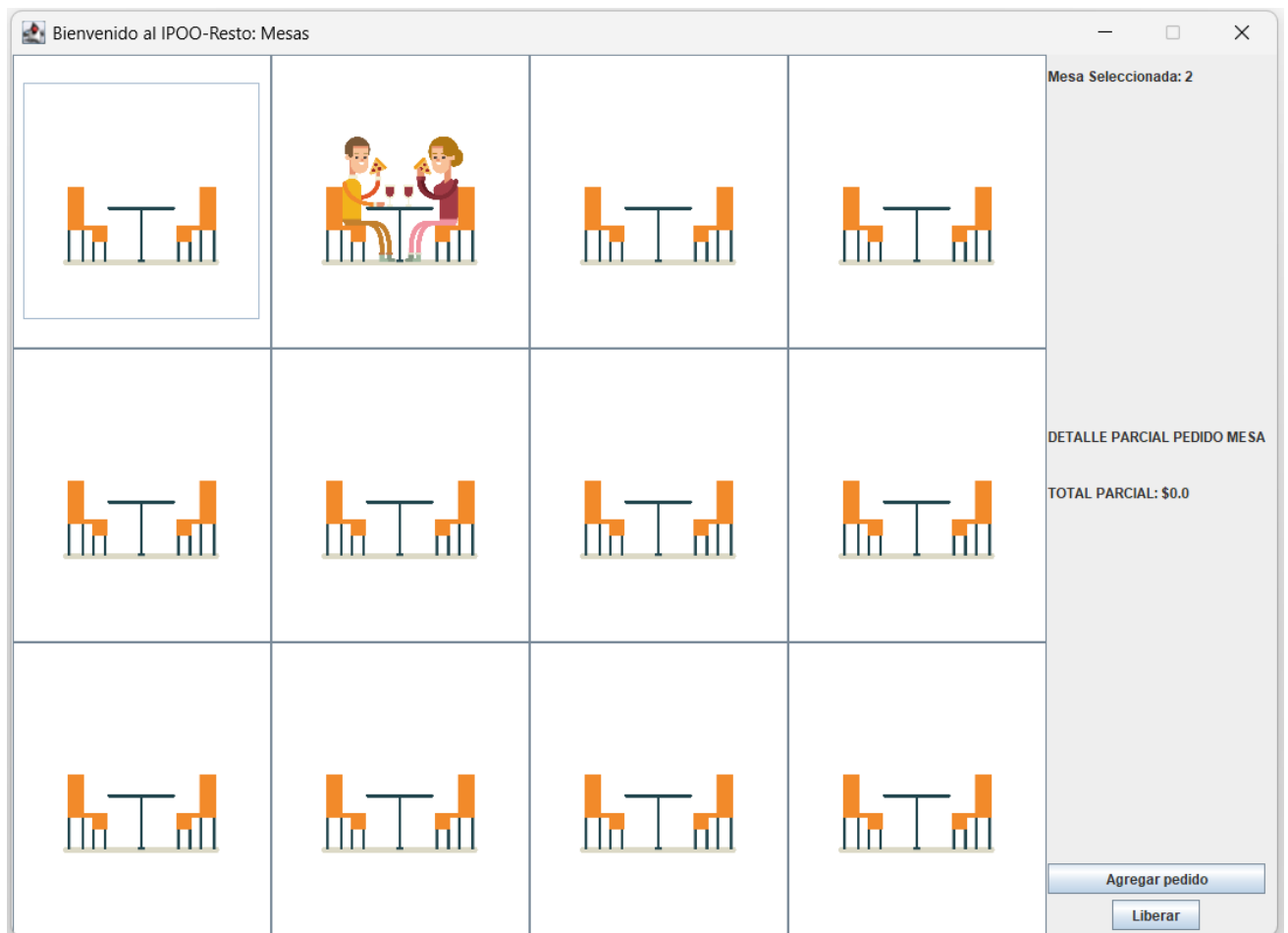
1. Inicialmente se debe mostrar la interfaz con todas las mesas del restaurante libres. Las mesas se disponen en un arreglo de botones que permite seleccionar una mesa en particular. En este punto únicamente es posible seleccionar una mesa.



2. Al seleccionar una mesa se mostrará a la derecha un *JPanel* panelMesa, que mostrará información relacionada con la mesa seleccionada. Inicialmente, panelMesa mostrará el número de mesa seleccionada y un único botón que permite ocupar la mesa. Las mesas se identifican con un único número que se asigna consecutivamente a partir de 1.



3. Al presionar el botón Ocupar, la mesa se ocupará, viéndose modificado el ícono correspondiente en el arreglo de mesas. Además, en el *JPanel* panelMesa se ocultará el botón Ocupar, se mostrará el detalle del pedido actual de la mesa y se mostrará un botón que permitirá cargar combos a la mesa conforme los comensales hagan su pedido. Además, se mostrará el botón Liberar, que permitirá cerrar la mesa cuando los comensales pidan la cuenta.



4. Al tener seleccionada una mesa ocupada es posible agregar combos al pedido mediante el botón Agregar Pedido, que permitirá que se visualicen los ítems disponibles del menú y sus precios. Mientras se estén agregando pedidos, se deberá impedir cambiar la mesa seleccionada.

Bienvenido al IPOO-Resto: Mesas

Mesa Seleccionada: 2

DETALLE PARCIAL PEDIDO MESA

TOTAL PARCIAL: \$0.0

Agregar pedido

Combo 1: Hamburguesa simple

Combo 2: Hamburguesa doble

Combo 3: Sandwich de pavita

Combo 4: Super pollo

Combo 5: Pollo frito

\$14000 quedan 10

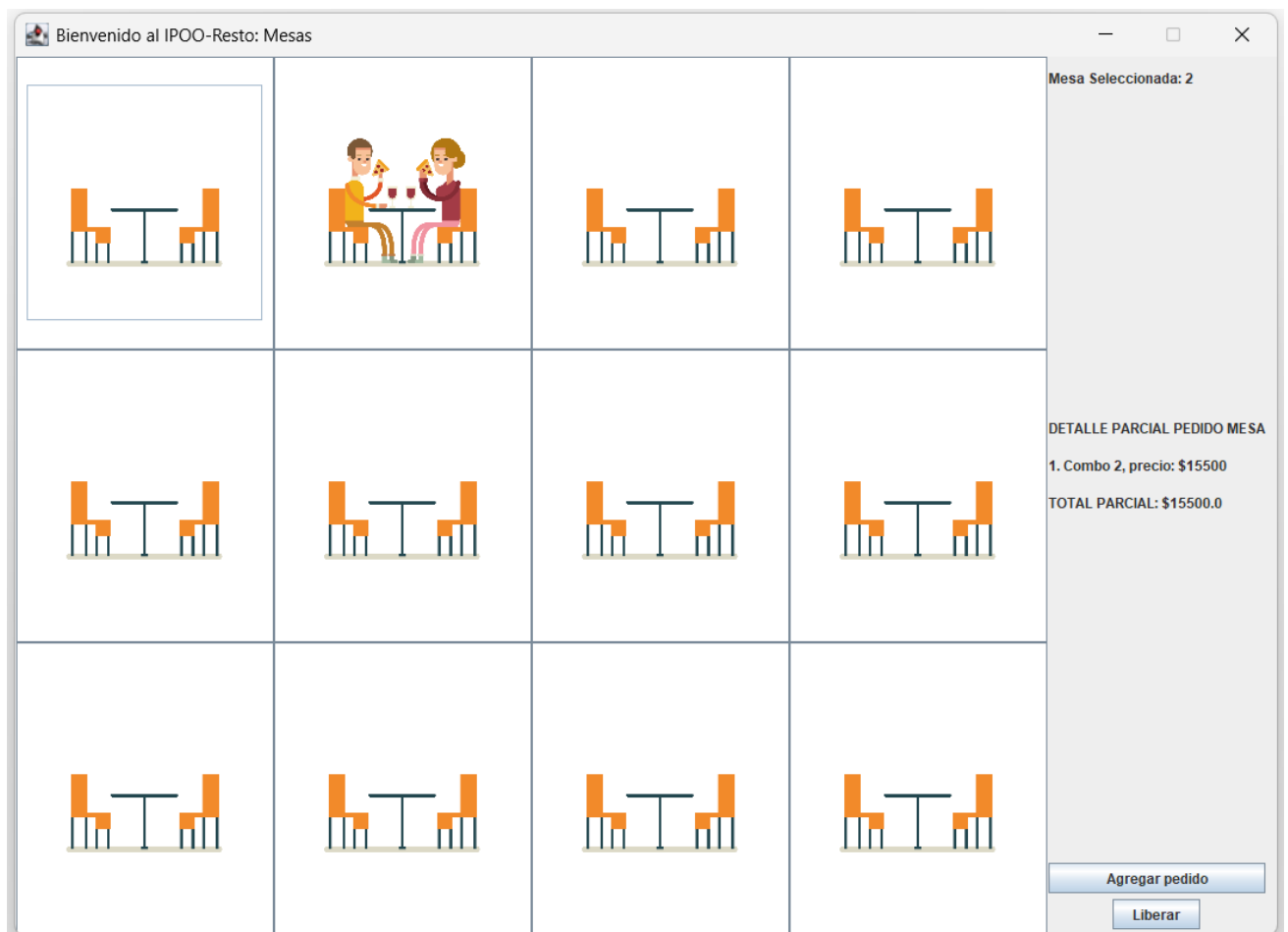
\$15500 quedan 20

\$14500 quedan 5

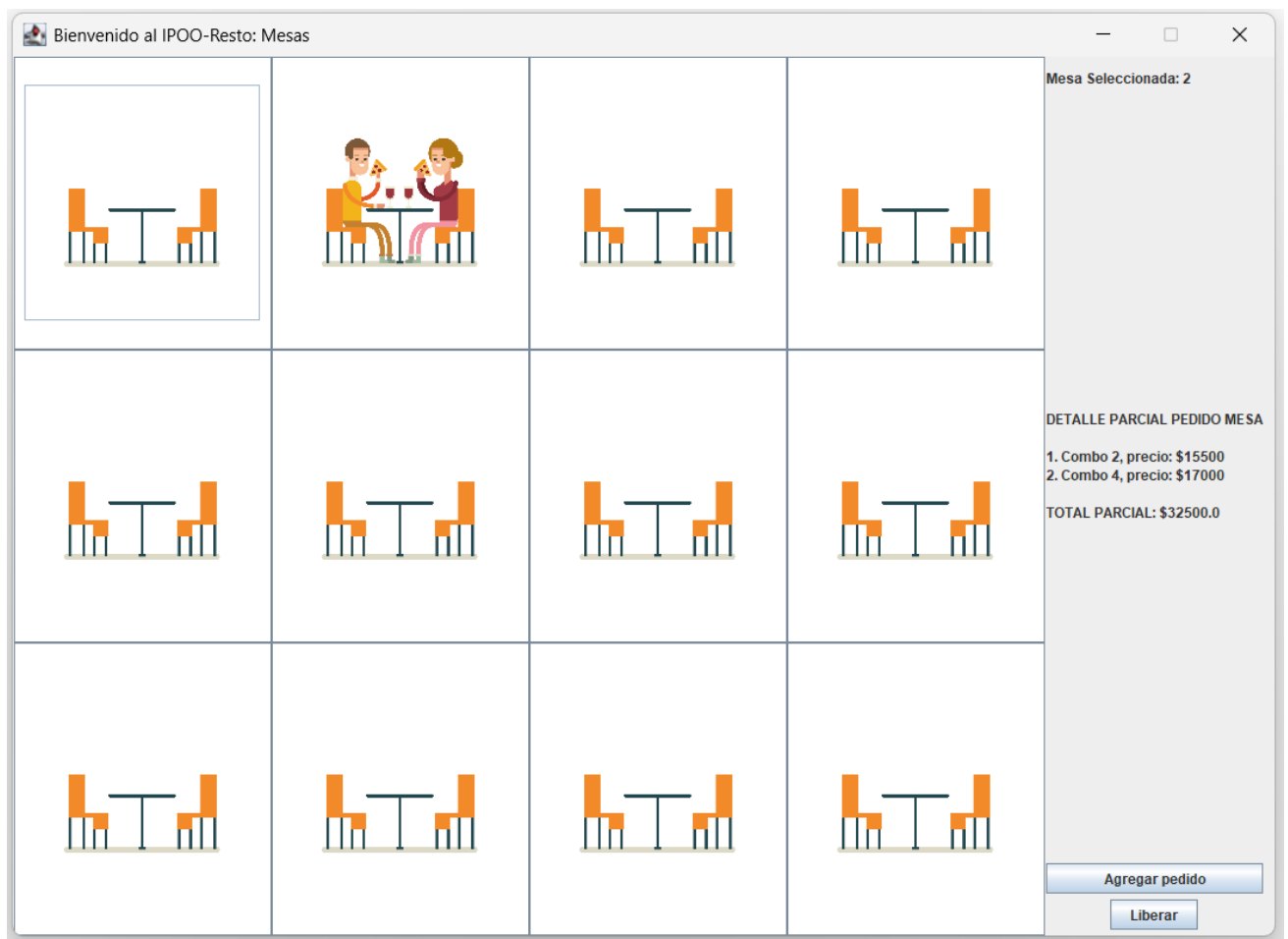
\$17000 quedan 10

\$15000 quedan 30

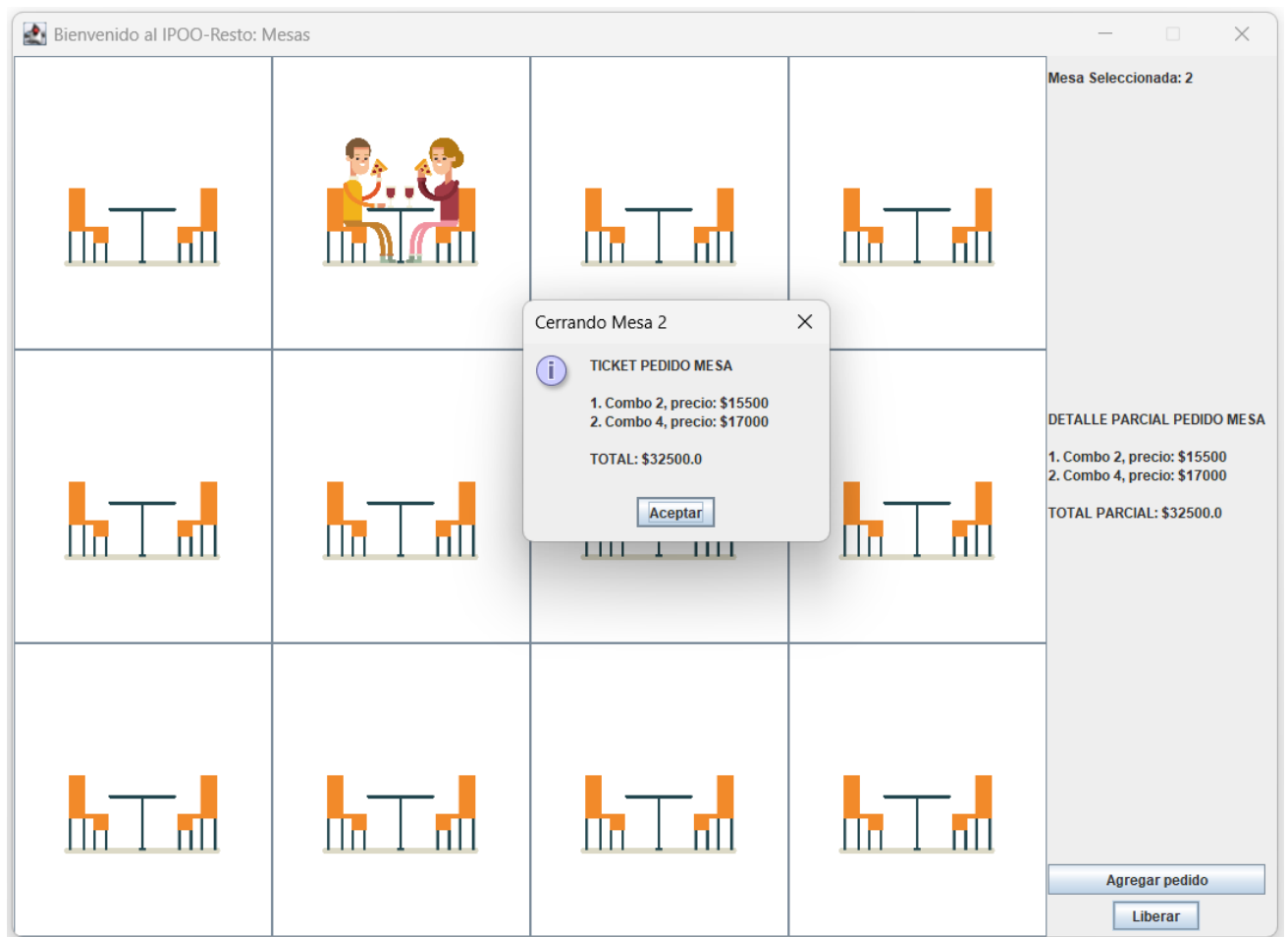
5. Al seleccionar un Combo, éste se carga a la mesa actualizando el stock del restaurante y el detalle del pedido de la mesa. Por ejemplo, si se carga un Combo 2 a la mesa 2 la interfaz se actualizará como sigue:



6. Cuando una mesa ocupada esté seleccionada, se puede seguir cargando combos o liberar la mesa. A continuación, se muestra la interfaz resultante de cargar en la mesa 2 un Combo 2 y un Combo 4.



7. Una mesa ocupada seleccionada se puede liberar haciendo click en el botón Liberar. Como resultado de esto se mostrará un cuadro de diálogo con el resumen de cuenta de la mesa y se pondrá la mesa de nuevo en estado libre volviendo a su estado inicial (paso 2).



Y al aceptar el cuadro de diálogo, se libera la mesa:



- Notar que, al agregar pedidos a las mesas, se va consumiendo el stock del restaurante, cuando el stock de un combo llega a cero debe deshabilitarse la compra del mismo.

 Combo 1: Hamburguesa simple	 Combo 2: Hamburguesa doble	 Combo 3: Sandwich de pavita	 Combo 4: Super pollo	 Combo 5: Pollo frito
\$14000 quedan 10	\$15500 quedan 19	\$14500 quedan 5	\$17000 quedan 9	\$15000 quedan 30

- Por otra parte, el restaurante no permite que en una misma mesa se haga un pedido de más de 10 combos. De modo que cuando una mesa alcanza esa cantidad máxima debe deshabilitarse la posibilidad de cargar más pedidos, dando opción únicamente de liberar la mesa.

Bienvenido al IPOO-Resto: Mesas

				Mesa Seleccionada: 12 DETALLE PARCIAL PEDIDO MESA 1. Combo 5, precio: \$15000 2. Combo 4, precio: \$17000 3. Combo 3, precio: \$14500 4. Combo 2, precio: \$15500 5. Combo 1, precio: \$14000 6. Combo 4, precio: \$17000 7. Combo 4, precio: \$17000 8. Combo 5, precio: \$15000 9. Combo 5, precio: \$15000 10. Combo 3, precio: \$14500 TOTAL PARCIAL: \$154500.0 Agregar pedido Liberar

El sistema está modelado respetando las siguientes clases:

ColCombos	Combo
<< atributos de instancia >> c: Combo[] cant: entero	<< atributos de instancia >> nombre: String; descripcion: String; precio: entero; cantidad: entero;
<< constructor >> ColCombos(cant:entero) <<comandos>> agregarCombo(e: Combo) << consultas >> cantCombos():entero obtenerCombo(pos:entero): Combo obtenerCombo(n:String): Combo obtenerPosCombo(n:String):entero precioTotal(): real	<< constructor >> Combo(n: String,p,c:entero, d:String) <<comandos>> vender() << consultas >> getNombre(): String getPrecio(): entero getCantidad(): entero getDescripcion(): String

El restaurante vende Combos de comida rápida. La clase *ColCombos* representa una colección de combos y es utilizada para representar tanto el menú de opciones en stock en el restaurante como el pedido de cada una de las mesas. Un *Combo* tiene un nombre, una descripción, un precio y la cantidad de unidades del combo disponibles en stock en el restaurant. Cada vez que se vende un combo, la cantidad en stock se reduce una unidad y sólo se podrá vender un combo mientras haya unidades en stock disponibles (el comando *vender()* requiere que haya unidades disponibles).

Resto	Mesa
<< atributos de instancia >> mesas: Mesa[] stockMenu: ColCombos	<< atributos de clase >> maxPedidosPorMesa: entero << atributos de instancia >> numero: entero pedidosMesa: ColCombos ocupada: boolean
<< constructor >> Resto(cant: entero, ColCombos stockMenu) << consultas >> obtenerMesa(numero: entero): Mesa cantMesas(): entero obtenerStockMenu(): ColCombos	<< constructor >> Mesa(n: entero) <<comandos>> ocupar() liberar() << consultas >> obtenerNumero(): entero obtenerPedido(): ColCombos estaOcupada(): boolean alcanzoMaximoPedidos(): boolean generarTicketCuenta(): String generarDetalleParcial(): String

La clase *Resto* representa el restaurante, que está compuesto por una tabla de Mesas numeradas correlativamente (a partir de 1) con todas sus componentes ligadas, y una colección de Combos (instancia de la clase *ColCombo*) representando el menú de opciones en stock.

- obtenerMesa(numero: entero): Mesa. Retorna la Mesa con el número pasado por parámetro.

Cada *Mesa* tiene un número, una instancia de la clase *ColCombos* representando el pedido, y un estado “ocupada” que indica si está ocupada o libre. Además, cada mesa puede tomar una cantidad de pedidos máximo, al alcanzar ese número no se pueden cargar más pedidos a la mesa.

- Al momento de crearse, se asigna un número a la mesa y su estado se configura en libre y sin pedidos.
- ocupar(). Si la mesa está libre, crea un nuevo pedido (*ColCombos*) y se configura como ocupada.
- liberar(). Se desliga la mesa del pedido (*ColCombos*) y se configura como libre.
- alcanzoMaximoPedidos(): boolean. Retorna verdadero si se alcanzó el límite de combos, y falso en caso contrario.

- `generarTicketCuenta(): String`. Arma un ticket con el detalle del pedido y total a abonar por los comensales.
- `generarDetalleParcial(): String`. Arma una lista parcial de los combos consumidos hasta el momento en la mesa y el subtotal a abonar.

Si la mesa está ocupada entonces estará asociada a una instancia de *ColCombos*: una colección de Combos que mantiene los pedidos realizados en la mesa hasta el momento y que ofrece servicios para agregar y consultar combos. Cada vez que se agregue un combo al pedido de la mesa, se deberá agregar dicho *Combo* a la colección *pedidosMesa* de la clase. La consulta *precioTotal(): real* de *ColCombos* retorna la suma de los precios de todos los combos en la colección hasta el momento.

GUIResto
<< atributos de instancia >> //Atributos de la aplicación resto: Resto numeroMesaSeleccionada: entero //Objetos Gráficos //Panel Menu panelMenu: JPanel boton: JButton[] etiqueta: JLabel[] //Panel Resto panelResto: Jpanel botonMesas: JButton[] //Panel Mesa panelMesa: JPanel etiquetaMesaSeleccionada: JLabel panelDetalle: JPanel etiquetaDetallePedido: JLabel botonAgregarItem: Jbutton panelOcuparDesocupar: Jpanel botonOcuparMesa: Jbutton botonDesocuparMesa: JButton
<< constructor >> GUIResto(r: Resto)

La clase *GUIResto* corresponde a la interfaz gráfica de la aplicación. El constructor *GUIResto(r: Resto)* configura la ventana, inicializa los atributos de la aplicación, la interfaz gráfica y registra los componentes.

Con la información brindada anteriormente, y la documentación que podrá encontrar en el código fuente, complete la implementación provista desarrollando en la clase *GUIResto* lo que se detalla a continuación:

1. Crear los botones *botonOcuparMesa* y *botonDesocuparMesa* e insertarlos en el panel correspondiente. Declarar, crear y registrar los oyentes para esos botones.
2. Completar los oyentes *OyenteOcuparMesa*, *OyenteLiberarMesa* y *OyenteCombo* para que la aplicación opere conforme a las funcionalidades descritas anteriormente.
3. Implementar el oyente *OyenteMesa* de manera que al seleccionar una mesa, actualice las etiquetas con la información correspondiente, y ajuste la visibilidad y el estado de los botones según si la mesa está ocupada o libre.
4. Implementar los métodos *inicializarPanelMenu()*, *armarBotones()* y *armarEtiquetas()* de acuerdo a las siguientes especificaciones:
 - *inicializarPanelMenu()*: Crea los botones correspondientes a los distintos combos del menú, registra sus oyentes y los inserta en el panel del menú. Además, genera las etiquetas con las descripciones de cada combo y las agrega al mismo panel.
 - *armarBotones()*: Configura los botones creados en el método anterior asignándoles la información específica de cada combo, como la imagen, la descripción y otros datos relevantes.

- *armarEtiquetas()*: Configura las etiquetas generadas en el método *inicializarPanelMenu()*, incorporando en cada una el precio del combo y la cantidad de unidades disponibles en stock.

SUGERENCIAS:

- Utilice como guía los algoritmos que se encuentran comentados en el código fuente.
- Se recomienda no modificar el código que les brinda la cátedra.